

CStats - Lab #7

April 21, 2026

Exercise 1.

The MLE of σ^2 in simple linear regression is $\widehat{\sigma^2} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$, but we instead use

$$\text{MSE} = \frac{1}{n-2} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

as an unbiased estimator of σ^2 . Intuitively, why do we divide by $n-2$ rather than n ? Why specifically the number 2?

Solution 1.

Dividing by n gives a biased (too small) estimate because the residuals $Y_i - \hat{Y}_i$ are computed using parameters $(\hat{a}, \hat{b})$ that were themselves chosen to *minimize* the sum of squared residuals. The fitted line is tuned to the data, so residuals are systematically smaller than the true errors $Y_i - \mu_{Y_i}$ would be.

The number 2 reflects the two degrees of freedom spent in estimating a and b . Equivalently, the residuals satisfy two linear constraints ($\sum e_i = 0$ and $\sum x_i e_i = 0$), so only $n-2$ of them are free to vary. Dividing by $n-2$ compensates for this loss of degrees of freedom and yields an unbiased estimator. More generally, for a model with p estimated parameters we divide by $n-p$.

Exercise 2.

The LINE assumptions for linear regression are Linearity, Independence, Normality, and Equal variance. The normality assumption is often stated as “the errors ε_i are normally distributed.” Why is this assumption needed, and at what stage of the analysis does it become essential? Is it needed to *compute* the least-squares estimates?

Solution 2.

Normality is *not* needed to compute least-squares estimates. The formulas for a and b follow from minimizing $\sum (Y_i - a - bx_i)^2$ and require no distributional assumption. By the Gauss-Markov theorem, least-squares estimates are the best linear unbiased estimators as long as errors have zero mean, constant variance, and are uncorrelated.

Normality becomes essential when we want to do *inference* — constructing confidence

intervals or performing hypothesis tests for a , b , μ_Y , or a new observation Y_{n+1} . The t-distribution results used for these CIs rely on the fact that $\hat{a}$ and $\hat{b}$, being linear combinations of normally distributed Y_i , are themselves normally distributed, and that $(n-2)S^2/\sigma^2 \sim \chi^2_{n-2}$. Without normality, the exact t-based CIs are not justified (though they are approximately valid for large n by the CLT).

Exercise 3.

A regression model reports $R^2 = 37.04\%$. What does this number mean intuitively? Does a high R^2 guarantee that the model is “good”? Does a low R^2 mean the model is useless?

Solution 3.

R^2 measures the proportion of the variability in the response Y that is explained by the linear relationship with the predictor X . Here, 37% of the variation in final grades is accounted for by quiz-average; the remaining 63% is due to other factors or noise.

A high R^2 does *not* guarantee a good model. The model could still violate LINE assumptions (e.g., non-linear pattern, heteroscedasticity), be overfit, or capture a spurious association. A high R^2 with wildly non-random residuals indicates the wrong model structure.

Conversely, a low R^2 does not make a model useless. In noisy domains (social sciences, financial returns) an R^2 of 10–20% may still reflect a real, useful signal. The slope b can be statistically significant and practically meaningful even when R^2 is modest. R^2 should be read alongside residual diagnostics and the p-value / CI for b , not in isolation.

Exercise 4.

Deep learning is presented as a three-step procedure: (1) define a set of functions, (2) assess the goodness of each function, (3) pick the best function. Map these three steps onto ordinary linear regression. What plays the role of the “function set,” the “loss,” and the “picking” step in linear regression?

Solution 4.

Step 1 — function set. In simple linear regression, the set is $\{f(x) = b + w x : b \in \mathbb{R}, w \in \mathbb{R}\}$. Each choice of (b, w) is one candidate function in the hypothesis class. This is directly analogous to the parameters θ of a neural network.

Step 2 — goodness of function. The loss is the sum of squared residuals, $L(w, b) = \sum_{n=1}^N (\hat{y}^n - (b + w x^n))^2$. Small L = good function.

Step 3 — pick the best function. For linear regression, we solve in closed form: setting $\partial L / \partial w = \partial L / \partial b = 0$ gives the least-squares formulas. For deep networks the closed form does not exist, so step 3 is performed iteratively using gradient descent. Linear regression is therefore a special (analytically tractable) case of the deep-learning recipe.

Exercise 5.

For linear regression we derived closed-form formulas for a and b by setting derivatives to zero. Why, then, do deep-learning courses spend so much time on gradient descent? Give at least two reasons why the closed-form trick does not extend to neural networks.

Solution 5.

First, the normal-equations solution works because the squared-error loss is quadratic in (a, b) and the model is linear in parameters; setting $\nabla L = 0$ produces a linear system. In a deep network, the composition of nonlinear activations (σ , ReLU, softmax) makes the loss a highly non-convex function of the weights. There is no algebraic way to solve $\nabla L(\theta) = 0$ in closed form.

Second, even if one could write down the equations, the sheer number of parameters (millions or billions) makes any direct matrix inversion computationally infeasible. Gradient descent scales to large parameter counts because each update only requires a gradient evaluation, not a matrix inverse.

Third, stochastic / mini-batch variants of gradient descent let us train on datasets too large to fit in memory, which closed-form least squares cannot do. For these reasons gradient descent (with backpropagation for efficient gradient computation) is the standard training tool for neural nets.

Exercise 6.

The lecture poses the following claim: “Every time we update the parameters by gradient descent we obtain θ that makes $L(\theta)$ smaller, so $L(\theta^0) > L(\theta^1) > L(\theta^2) > \dots$.” Is this statement correct? If not, under what conditions can it fail?

Solution 6.

The statement is *not* correct in general. Gradient descent only guarantees decrease of the loss for an *infinitesimally small* step. With a finite learning rate η , several failure modes can occur:

1. **Learning rate too large:** The update overshoots the minimum and lands at a point with higher loss. In the worst case the iterates diverge.
2. **Non-convex landscape:** Even with a reasonable η , momentum or stochastic gradients can temporarily increase the loss while escaping saddle points or shallow local minima.
3. **Stochastic gradient descent:** Gradients are computed from mini-batches and are noisy estimates of the true gradient, so L fluctuates rather than decreasing monotonically.

Monotonic decrease only holds with a sufficiently small step size on a smooth, well-behaved loss. In practice we accept occasional increases in exchange for faster overall

progress.

Exercise 7.

In the Pokémon CP example, polynomial models of degree 1 through 5 gave these errors:

degree	Training err.	Testing err.
1	31.9	35.0
2	15.4	18.4
3	15.3	18.1
4	14.9	28.2
5	12.8	232.1

Explain, intuitively, why training error keeps dropping while testing error eventually explodes. Which model would you deploy, and why?

Solution 7.

Increasing the polynomial degree enlarges the hypothesis class: every degree- k polynomial is also a degree- $(k+1)$ polynomial with the top coefficient set to zero. So the best achievable training error can only get smaller (or stay the same) as complexity grows. This is why the training column is monotonically decreasing.

Testing error measures how well the chosen function predicts *new* data. A highly flexible model can chase the random noise in the training set — passing close to each training point at the cost of wild behavior between them. Those wiggles generalize poorly, so testing error eventually rises sharply. This is *overfitting*.

I would deploy the degree-2 or degree-3 model. They give essentially the same testing error as the more complex models in the sweet spot, but are simpler, more interpretable, and less likely to overfit if the training set changes. The degree-5 model has a training error of 12.8 but a testing error of 232.1, a catastrophic generalization gap.

Exercise 8.

Regularization adds a penalty to the loss:

$$L = \sum_n \left(\hat{y}^n - (b + \sum_i w_i x_i) \right)^2 + \lambda \sum_i w_i^2.$$

- (a) Why does this encourage “smoother” functions? (b) Should the bias b also be penalized? (c) How should one pick λ ?

Solution 8.

(a) The penalty $\lambda \sum w_i^2$ rewards small weights. Consider changing the input by Δx_i : the output changes by $\sum w_i \Delta x_i$. Small w_i means the output is less sensitive to perturbations in the input — i.e., the function varies slowly, i.e., it is smoother. Smoother functions are less able to chase noise, so regularization combats overfitting.

(b) No. The bias b shifts the whole function vertically; it does not affect smoothness. Penalizing b would bias the predictions toward zero for no statistical reason, and could

hurt performance without offering any regularization benefit.

(c) λ is a hyperparameter, chosen by validation. Fit models with several candidate λ s on a training set, evaluate them on a held-out validation set (or via cross-validation), and pick the λ that minimizes validation error. In the Pokémon example, $\lambda = 100$ gave the best test error (11.1); $\lambda = 0$ overfits, while $\lambda = 100000$ over-smooths and underfits.

Exercise 9.

Draw the simplest possible neural network that implements ordinary linear regression $\hat{y} = b + w x$. How many layers does it have? What is the activation function? What does this tell you about the relationship between linear regression and deep learning?

Solution 9.

The network has:

- one input neuron holding x ,
- one output neuron that computes $\hat{y} = w x + b$,
- *no* hidden layers,
- the identity function as activation (i.e., no nonlinearity).

This is sometimes called a “single-layer perceptron with linear activation.” The number of trainable parameters is exactly 2 (one weight and one bias), matching linear regression.

The moral is that linear regression sits at one end of a continuous spectrum: add hidden layers and nonlinear activations and you get a multilayer perceptron; keep stacking and you arrive at modern deep networks. All three are trained by the same three-step recipe (model + loss + optimizer), differing only in the richness of the function class and the optimization method.